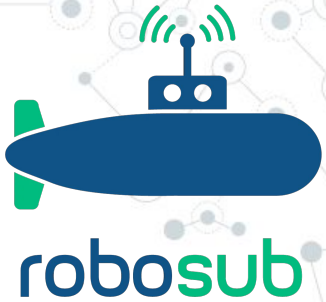
Software Team Deep Dive

Robosub 2026: The Software Challenge



Goal: We are building the software that allows an underwater robot to operate without someone controlling every movement.

The robot needs to:

- Understand what it's sensors are telling it
- Figure out where it is
- Decide what it needs to do
- Move and interact with environment

SEE -> UNDERSTAND -> DECIDE -> ACT

RoboSub 2026 Competition

RoboSub 2026 gave us a good example of the kinds of problems our autonomous underwater vehicle has to solve.

!!! Tasks are similar each year but can change

Let’s take a deep dive!



2026 Course Overview

1. Begin Assessment

Get through the gate and choose a role.

2. Avoid Debris

Navigate through the slalom.

3. Recon

Identify bins and deliver markers.

4. Deploy

Fire torpedoes at a target.

5. Restore

Collect and deliver objects.

6. Return Home

Find the way back through the gate.

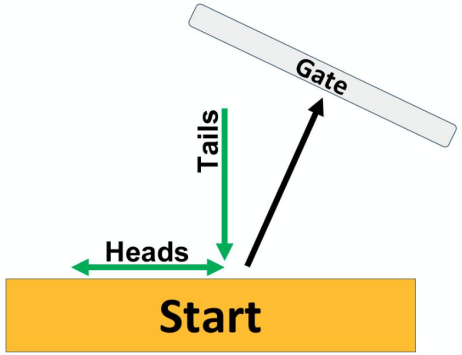
Task 1: Getting Through the Gate

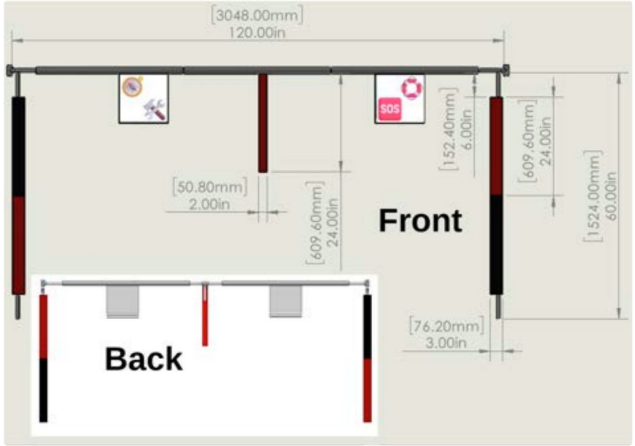
The robot has to:

- Find the gate
- Identify the role indicator
  - Choose the correct side and remember its role
- Maintain its direction and depth while passing through

Software Challenge:

Turn camera data into a reliable decision about where the robot needs to go.

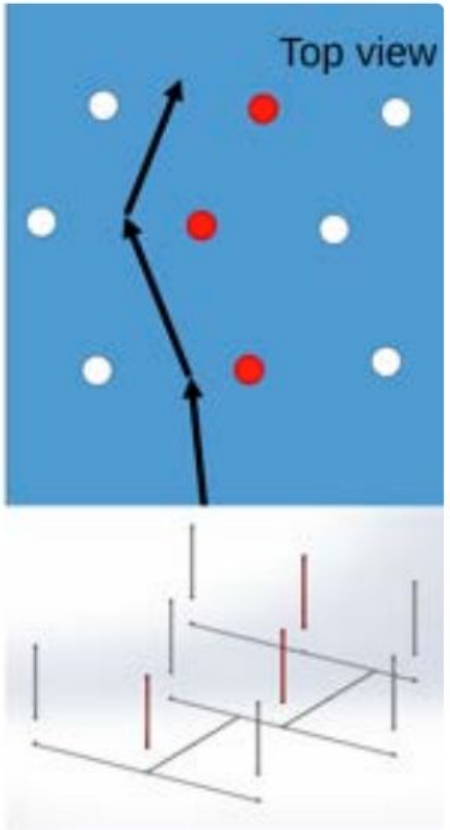




Task 2: Avoid Debris (Navigate Slalom)

The robot has to navigate a series of pipes:

- Recognize the slalom
- Maintain the right depth
- Correct its course as it moves
- Keep track of which side of the red pipe it should be on.



Software Challenge:

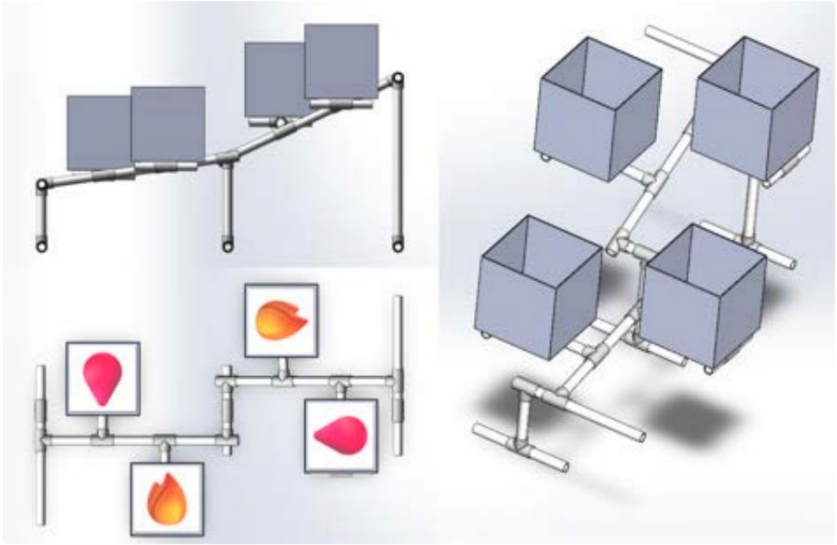
Seeing an obstacle is one thing but continuously adjusting the vehicle around it is harder.

Task 3: Dropper Bins

The robot has to inspect a group of bins and decide where to place its markers.

It need to:

- Find the bins
- Identify what's inside them
- Hold its position
- Drop a marker
- Potentially interact with magnetic lights



Software Challenge:

No longer just navigating, it has to identify and perform an action based on what it sees.

Task 4: Fire Torpedo

The robot has to locate a target and fire a torpedo through an opening.

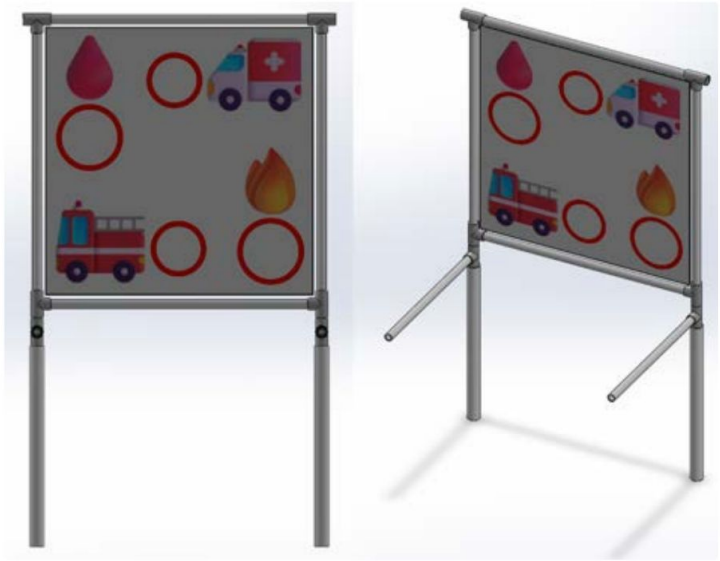
It need to:

- Find the target.
- Get into the right position
- Hold that position
- Aim at opening
- Fire!

Software Challenge:

Positioning and orientation is key.

Task

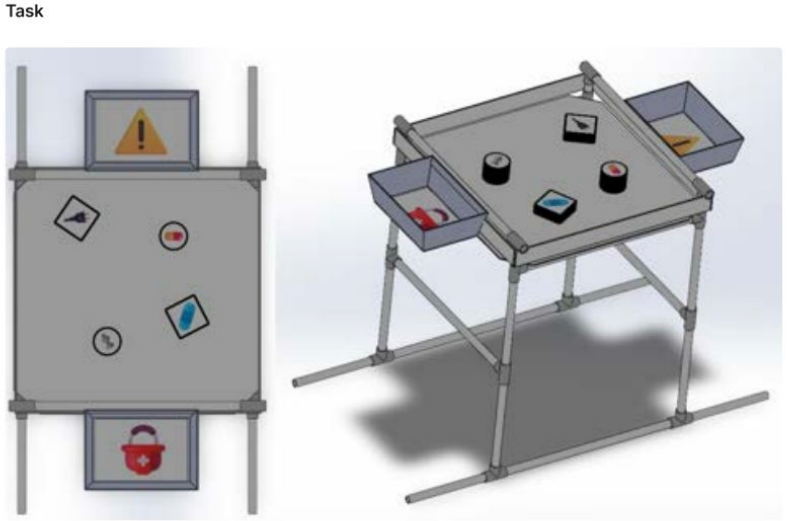


Task 5: Pick up and Surfacing

This is one of the more complicated tasks.

It need to:

- Find the recovery area
  - Acoustic pinger localization
- Surface in the correct area
- Identify the objects
- Pick them up
  - Might have to surface with it as well?
- Put them in the appropriate basket



Software Challenge:

Navigation, vision, and positioning all have to work together

Task 6: Return Home

After completing all tasks, AUV has to return back through starting gate

It need to:

- Determine where home is
- Navigate there
- Maintain control
- Pass back through the gate while underwater

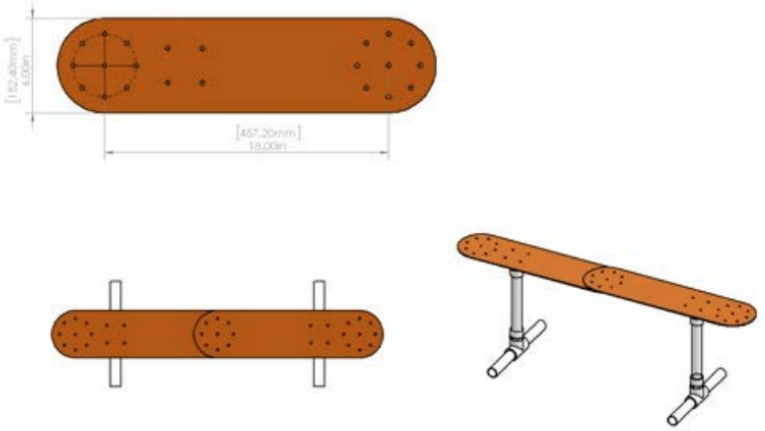


Figure : Path marker

Software Challenge:

Knowing when and where to go back.

What do these tasks have in common?

Looking at the tasks, we keep running into the same problems:

Where am I?
Position, depth, heading

What am I looking at?
Gates, pipes, bins, targets, objects

What should I do next?
Decide which action makes sense

How do I actually do it?
Move, hold position, turn, fire, grab, release



That's the software we need to build.

Software Architecture

We don't want one giant program controlling the entire robot. Instead, different parts of the software handle different jobs:

Perception
What does the robot see?

Localization
Where is the robot?

Navigation
How should it get there?

Mission Logic
What should it do next?

Vehicle Control
How do we make the robot move?

OUR Software Architecture

Hardware

Jetson Nano
Runs the higher-level software: ROS 2, computer vision, navigation, and mission logic.

Pixhawk Flight Controller
Contains sensors like gyroscopes, accelerometers, and compasses. Runs ArduSub Software.





Software

QGroundControl
Essentially a diagnostic tool for testing. Used for configuration, calibration, telemetry, and testing.

ArduSub
Process sensor data, interpret commands, and control underwater vehicle motors and lights

MAVROS + MAVLink
Connects our ROS 2 software to the Pixhawk.

Pixhawk + ArduSub

Pixhawk is the flight controller.



ArduSub is the autopilot software running on it.

Our high-level software shouldn't have to worry about things like:

- Stabilizing the vehicle
- Constantly correcting the thrusters
- Handling low-level control

Instead we give our Pixhawk a command like:

- “Move Forward”
- “Hold this Depth”
- “Roll Right”

And ArduSub will handle the rest of the vehicle control.

Jetson Nano

The Jetson is the computer running our high-level software

ArduSub is the autopilot software running on it.

This is where we can:

- Process camera data
- Run computer vision frameworks
- Run ROS 2
- Keep track of the mission
- Make decisions
- Send commands to the Pixhawk



Jetson: decides **what** the robot should do
Pixhawk: controls **how** the robot does it

ROS 2

ROS 2 lets us break the system into separate programs called nodes.



For example:

Camera Node

⬇

Object Detection Node

⬇

Navigation Node

⬇

Mission Logic Node

Each part can be developed and tested separately while still being able to communicate with the rest of the system. The goal is to give everyone a clear place to plug their code into the robot.

MAVROS + MavLink (ROS Cont.)

Our ROS 2 software and the Pixhawk don't speak the same language.

MAVLink is the communication protocol used to exchange information with the flight controller.

MAVROS acts as the bridge between our ROS 2 software and MAVLink.

For example:

ROS 2

⬇

MAVROS

⬇

MAVLINK

⬇

PixHawk

Computer Vision



Teaching our AUV to recognize what it sees.

We need software that can turn:

Image → Detection → Useful information

For example:

ZED2i

↓

Image / Depth

↓

OpenCV / YOLO

↓

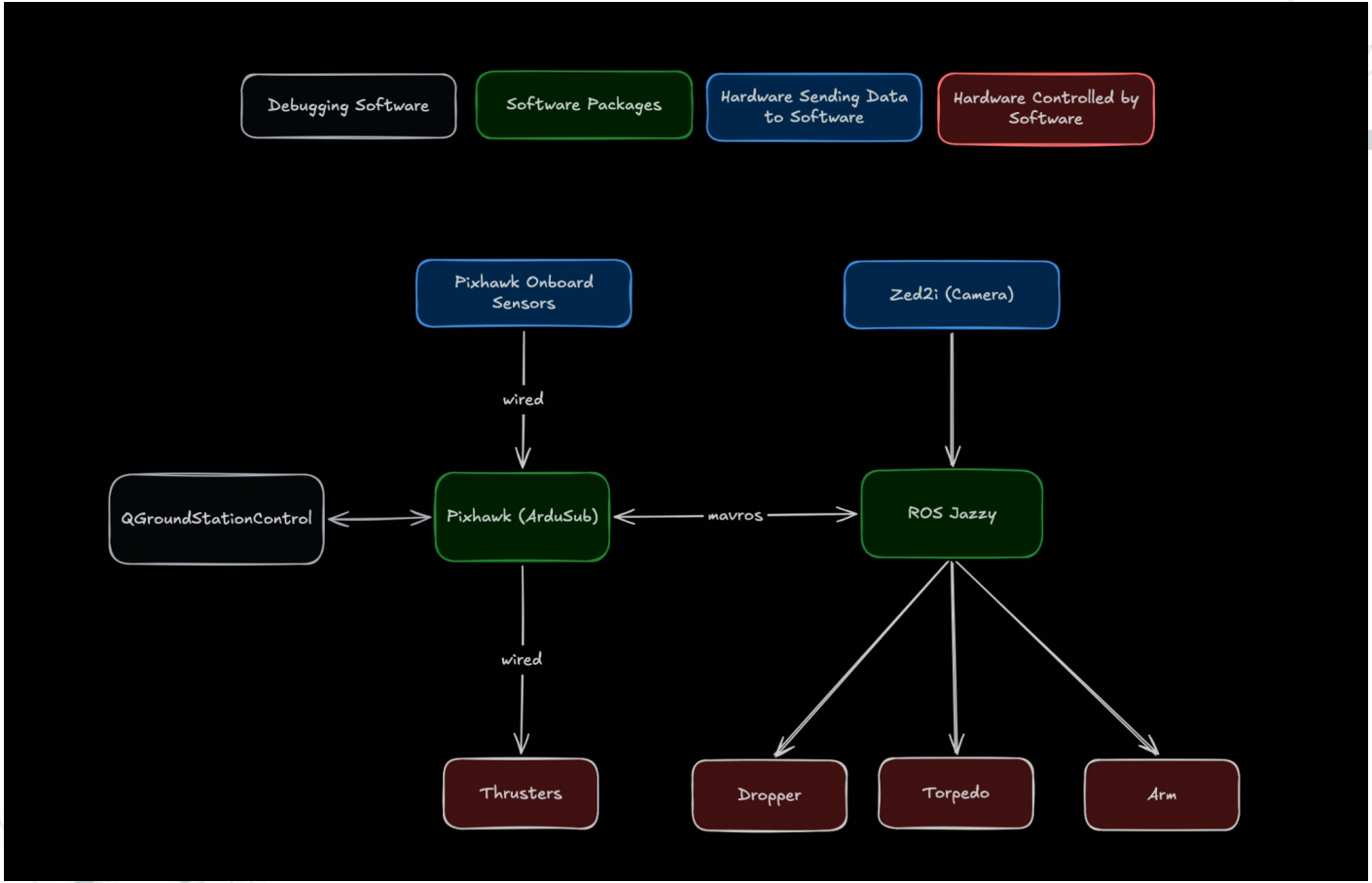
"I found the gate"

↓

ROS 2

Example Software Architecture Diagram

WIP



Example: What Happens When the Robot Finds a Gate?



Let's say the camera sees a gate.

1. Camera
Records an image of the gate

2. Computer Vision
Detects the gate and interprets what it is.

3. ROS 2
Passes that information to the rest of the system.

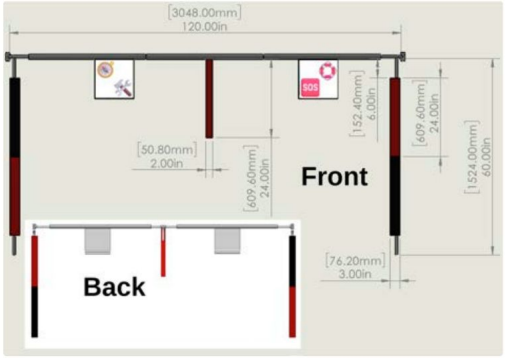
4. Mission Logic (Code on Jetson)
Decides that the gate is the current objective.

5. Navigation (ArduSub Autopilot)
Figures out how the vehicle should approach it.

6. Pixhawk
Controls the vehicle as it moves.

7. Sensors
Tell us what happened.

Then the process repeats.



So What Does Each Software Member Work On?

Vehicle Interface

Pixhawk, ArduSub, MAVLink, MAVROS

ROS 2

Nodes, topics, services, actions, system interfaces

Computer Vision

ZED2i, OpenCV, YOLO, object detection

Navigation

Position, depth, heading, station keeping

Mission Logic

State machines, task decisions, recovery behavior

Integration & Testing

SITL, testing, logs, debugging, end-to-end integration

Areas of Intrigue

There’s still a lot we don’t know

We have a general idea of where we’re going, but we still don’t have the entire system figured out.

Computer Vision
Work with the ZED2i, OpenCV, YOLO, and object detection.

ROS 2 & Systems Integration
Build nodes, communication, and interfaces between the different parts of the robot.

Navigation & Localization
Explore how the AUV can determine its position, heading, depth, and movement underwater.

Autonomy & Mission Logic
Develop the decision-making that tells the robot what to do and how to respond when things don't go as planned.

Simulation & Vehicle Control
et an AUV running in simulation and experiment with ArduSub, Pixhawk, and MAVROS.

There's a lot of setup to do before we can start building the full autonomy system.

Pixhawk / QGroundControl

- Connect to the Pixhawk
- Install ArduSub onto it/
- Learn how QGroundControl communicates with the vehicle
- Test basic telemetry and commands

Simulation

- Set up Software In The Loop (SITL)
- Run ArduSub in simulation
- Experiment with controlling a simulated vehicle

Computer Vision

- Set up the ZED2i software
- Get camera data working
- Experiment with YOLO / object detection
- Begin testing models on the Jetson

ROS 2

- Set up ROS 2 development environments
- Create basic nodes and communication
- Start figuring out how ROS 2 will interact with the vehicle

Git / GitHub or GitLab

- Create the team's repository
- Establish a basic project structure

Resources

[RoboSub 2026 Handbook]